UPES, Dehradun

School of Computer Science
Artificial Intelligence Cluster

ASYMPTOTIC NOTATIONS: BIG O, BIG OMEGA, BIG THETA

Mid-Semester Assignment Report

Course:	Design and Analysis of Algorithms (CSEG3053)
Student:	Yashvardhan Singh
SAP ID:	590021777
Batch:	B17
Semester:	III (Aug–Dec 2026)
Submitted to:	Kunwar Siddharth, Assistant Professor
Date:	15 October 2026

Prepared for submission as a soft-copy PDF and printed hard copy.

Abstract

Asymptotic notation is a mathematical language used to describe how an algorithm's resource requirements grow as the input becomes large. This report explains three central notations: Big O, Big Omega, and Big Theta. Big O gives an asymptotic upper bound, Big Omega gives a lower bound, and Big Theta gives a tight bound when upper and lower bounds have the same order of growth. The report introduces the idea using simple examples, states formal definitions, and demonstrates the analysis of a loop-based algorithm. A Python animation plots constant, logarithmic, linear, linearithmic, and quadratic growth as the input size increases. The key lesson is that asymptotic analysis helps compare algorithm scalability independently of a particular computer, while constants and lower-order terms still matter for small inputs and practical performance.

1. Introduction

An algorithm is a sequence of steps used to solve a problem. When choosing between two algorithms, it is useful to know how their running time or memory use changes as the input grows. Measuring seconds on one laptop is not enough: the result depends on hardware, programming language, implementation, and system load. Asymptotic analysis instead studies the growth trend as input size n becomes large.

This topic is part of Design and Analysis of Algorithms. It is useful when selecting searching, sorting, graph, and data-processing methods. For example, an algorithm that performs roughly n^2 operations may be acceptable for 20 items but much slower than an n log n algorithm for 100,000 items. Asymptotic notation gives a concise way to describe and compare such trends.

2. Intuition and Core Idea

Big O describes an upper bound, **Big Omega** describes a lower bound, and **Big Theta** describes a tight bound. They are not automatically synonyms for worst, best, and average cases. Those cases describe which input or scenario is measured, while asymptotic notation describes the growth bound of the resulting function.

In simple words: O(g(n)) means the function eventually grows no faster than a constant multiple of g(n). Omega(g(n)) means it eventually grows at least as fast as a positive constant multiple of g(n). Theta(g(n)) means it is bounded above and below by constant multiples of g(n).

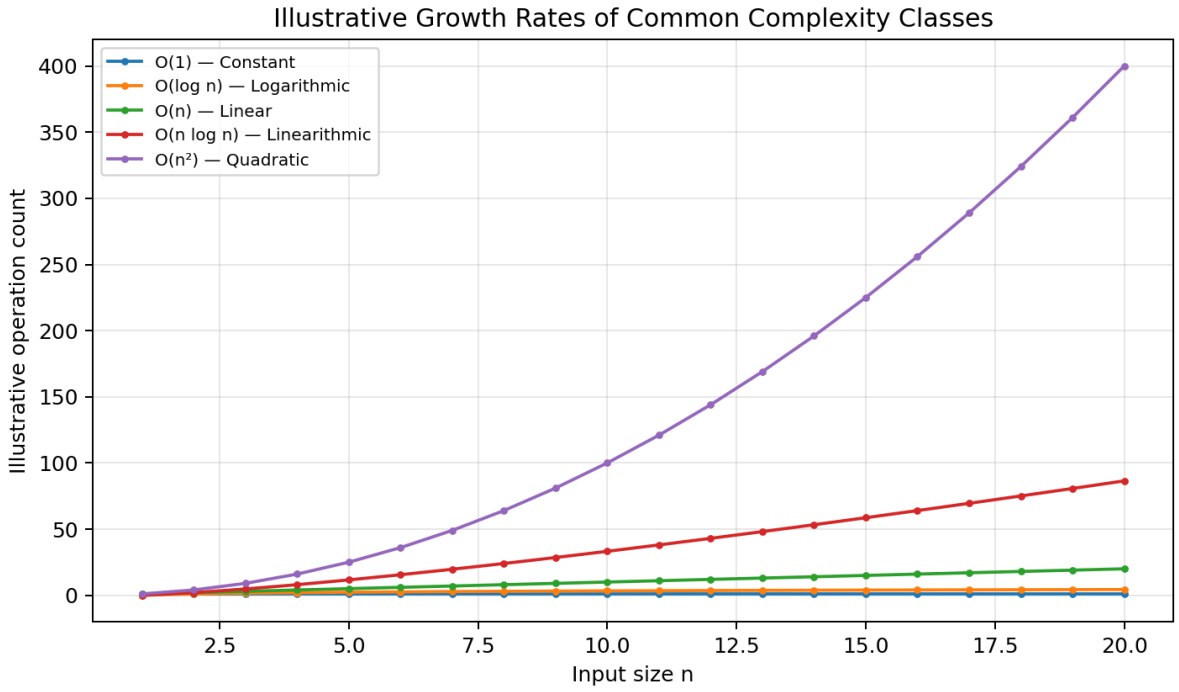


Figure 1. Illustrative growth rates. The plotted values compare trends, not actual timings from a particular computer.

3. Mathematical Formulation

3.1 Big O: asymptotic upper bound

f(n) = O(g(n)) if there exist positive constants c and n■ such that 0 ≤ f(n) ≤ c g(n) for all n ≥ n■.

Example: f(n) = 3n² + 5n + 2 is O(n²), because for sufficiently large n, the lower-order terms can be bounded by a constant multiple of n².

3.2 Big Omega: asymptotic lower bound

f(n) = Ω(g(n)) if there exist positive constants c and n■ such that 0 ≤ c g(n) ≤ f(n) for all n ≥ n■.

The same polynomial 3n² + 5n + 2 is Ω(n²), because it is at least 3n² for positive n.

3.3 Big Theta: tight asymptotic bound

f(n) = Θ(g(n)) if there exist positive constants c■, c■, and n■ such that 0 ≤ c■g(n) ≤ f(n) ≤ c■g(n) for all n ≥ n■.

Since 3n² + 5n + 2 is both O(n²) and Ω(n²), it is Θ(n²).

4. Algorithm / Procedure

The following pseudocode counts how often an inner statement runs. The outer loop executes n times and the inner loop executes n times for each outer iteration, so the total count is n × n = n².

```
def count_operations(n):
    count = 0
    for i in range(n):
        for j in range(n):
            count += 1
    return count
```

For positive n, the function returns n². Therefore, its operation count is Θ(n²), and consequently it is also O(n²) and Ω(n²). Theta is the most informative of these descriptions because it gives a tight bound.

5. Worked Example (Step by Step)

Consider f(n) = 3n² + 5n + 2.

1. Identify the terms: 3n², 5n, and 2.

2. For large n, the quadratic term grows faster than the linear and constant terms.

3. For n ≥ 1, n ≤ n² and 1 ≤ n², so f(n) = 3n² + 5n + 2 ≤ 3n² + 5n² + 2n² = 10n². This proves O(n²), with c = 10 and n■ = 1.

4. Also f(n) ≥ 3n² for n ≥ 1, proving Ω(n²), with c = 3 and n■ = 1.

5. Combining both bounds gives f(n) = Θ(n²).

n	f(n) = 3n² + 5n + 2	n²
1	10	1
2	24	4
5	102	25
10	352	100

These are illustrative values. Asymptotic notation describes growth for sufficiently large inputs, not exact equality between functions.

6. Animation / Software Demonstration

The demonstration is implemented in Python using NumPy and Matplotlib. It animates increasing input size and compares five functions: 1, log■ n, n, n log■ n, and n². The animation shows that quadratic growth eventually rises much faster than the other plotted functions.

Files: main.py, requirements.txt, growth_rates_animation.gif. **How to run:** Install Python 3, run “pip install -r requirements.txt”, then run “python main.py”. The program displays the animation and saves a GIF. The supplied GIF can also be opened directly.

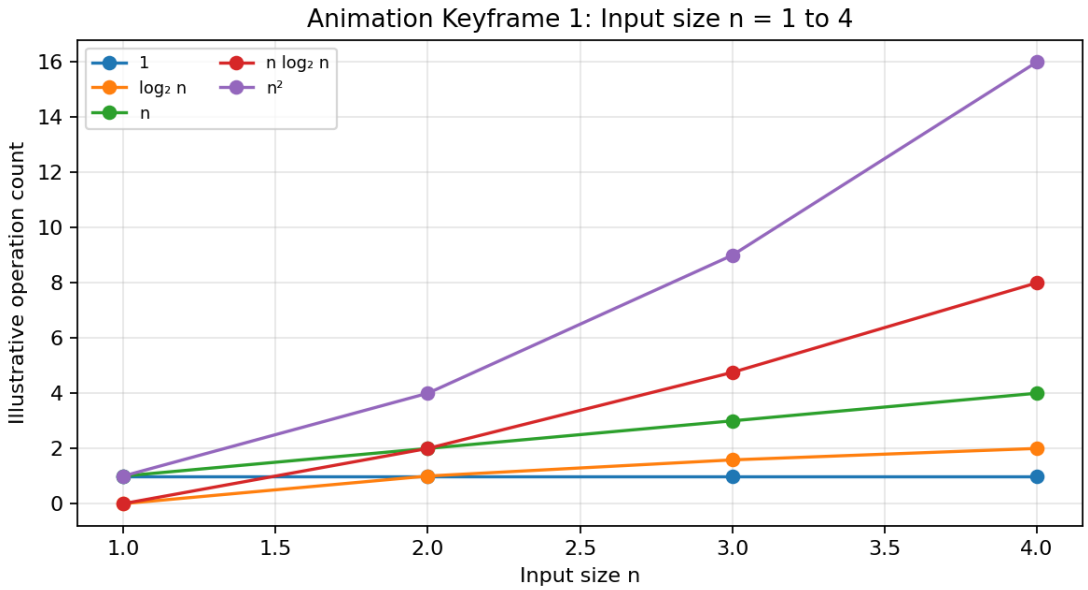


Figure 2. Keyframe 1: small input size.

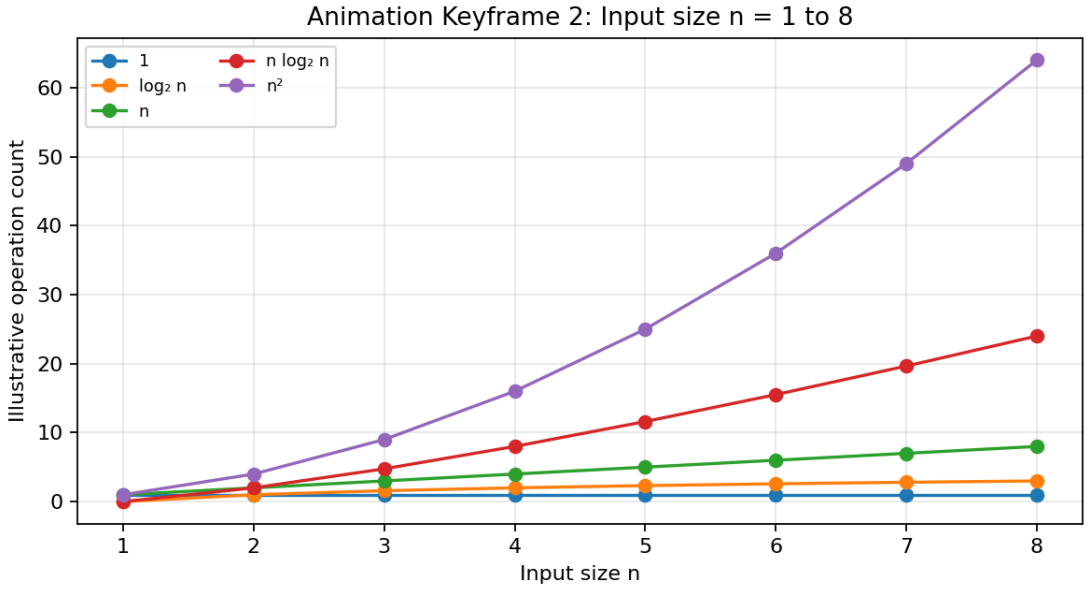


Figure 3. Keyframe 2: early growth.

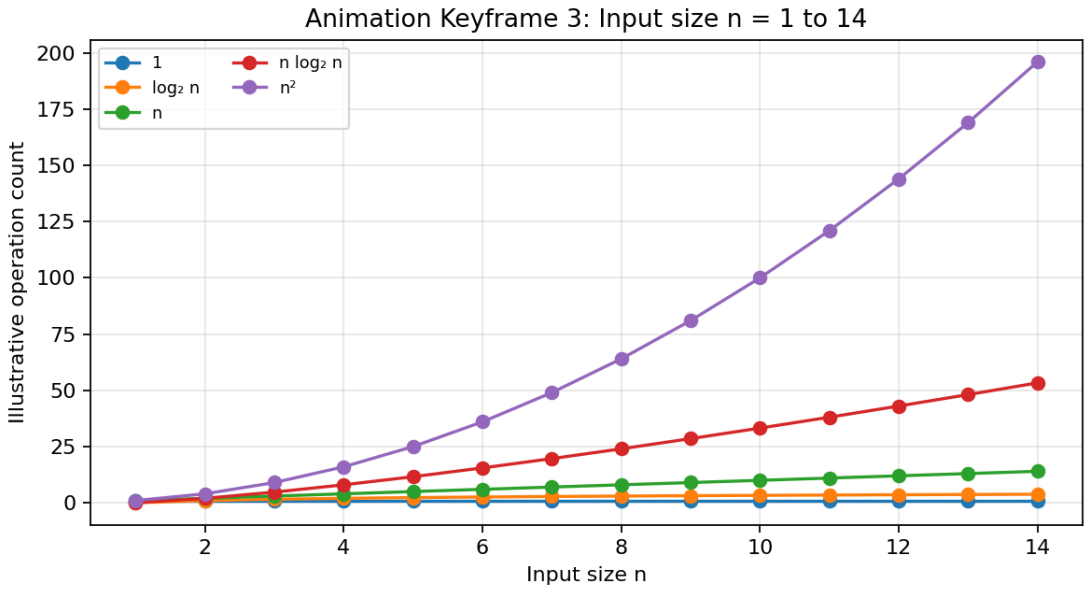


Figure 4. Keyframe 3: increasing separation.

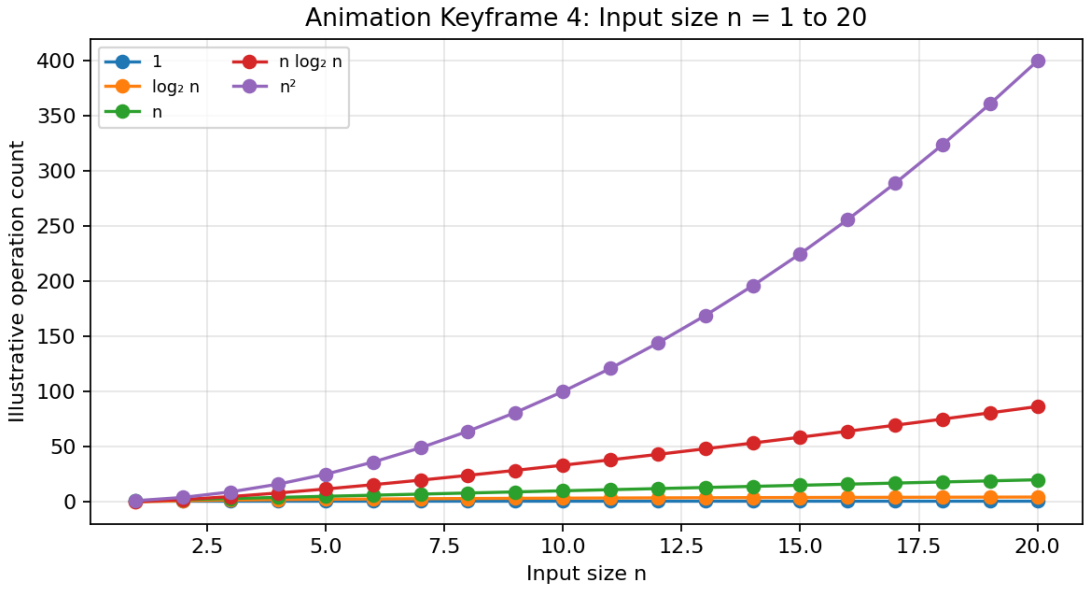


Figure 5. Keyframe 4: larger input size.

7. Analysis

Criterion	Asymptotic notation	Exact benchmarking
Purpose	Describes growth trend	Measures execution time
Strength	Hardware-independent comparison	Reflects a real implementation
Limitation	Hides constants and lower-order details	Depends on hardware and environment
Best use	Predicting scalability	Testing practical performance

Typical growth classes, from slower to faster for sufficiently large n, are O(1), O(log n), O(n), O(n log n), and O(n²). This ordering helps explain why merge sort, with Θ(n log n) time, usually scales better than simple quadratic sorting methods for large inputs. Constants, memory use, and input characteristics can still affect practical choices.

8. Applications

1. Searching: Linear search has O(n) worst-case time, while binary search on sorted data has O(log n) time.

2. Sorting: Merge sort has $\Theta(n \log n)$ time, while simple nested-loop sorting approaches commonly have $\Theta(n^2)$ time.

3. Data processing: Complexity analysis helps estimate how processing costs change as datasets become larger.

9. Conclusion

This report introduced Big O, Big Omega, and Big Theta as upper, lower, and tight asymptotic bounds. A polynomial example showed how to prove a tight bound, while a nested-loop program demonstrated quadratic operation growth. The animation compared common growth rates visually. The main conclusion is that asymptotic analysis is a useful tool for reasoning about scalability, but it should be combined with practical testing when choosing an implementation.

Declaration of AI Tool Use

Tools used: ChatGPT assisted with structuring and drafting explanations, and generating example code and figures.

What for: Initial drafting, explanation of asymptotic bounds, and code/figure preparation.

Approximate share of AI-generated content: Substantial. Review and revise the report to match course requirements and your own work before submission.

I confirm that I have reviewed the report and will ensure that the submitted work follows the course policy.

Signature: _____ Date: _____

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[2] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.

[3] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.